Faculty of Computers and Artificial Intelligence

Cairo University

CS251: Intro to Software Engineering

Dr.Mohammad El-Ramly

Assignment 1

Presented by

Name	ID	Section	Email	Team leader's phone
Yasmin Mohamed Nabil	20240658	S20	20240658@stud.fci-cu.edu.eg	01095955995
Lojain Mohammed Abu Elhassan	20240435	S20	20240435@stud.fci-cu.edu.eg	
Mario Nasser Fawttzy	20240439	S20	20240439@stud.fci-cu.edu.eg	

CS251: Phase 1 – Digital Fin
Project: Budget wise
Software Requirements Specifications

Contents

Document Purpose and Audience

What is this document?

This document explains **how the system will be built**.
It includes:

- system architecture
- design decisions
- components and modules
- class diagrams and technical details

It focuses on **implementation and design**

Target Audience: it is intended for several groups involved in the development, including:

1. Developers
2. Software Architects
3. Technical Team
4. System Designers
5. Testers (for understanding structure)

System Models

I. Architecture Diagram

Software Architecture

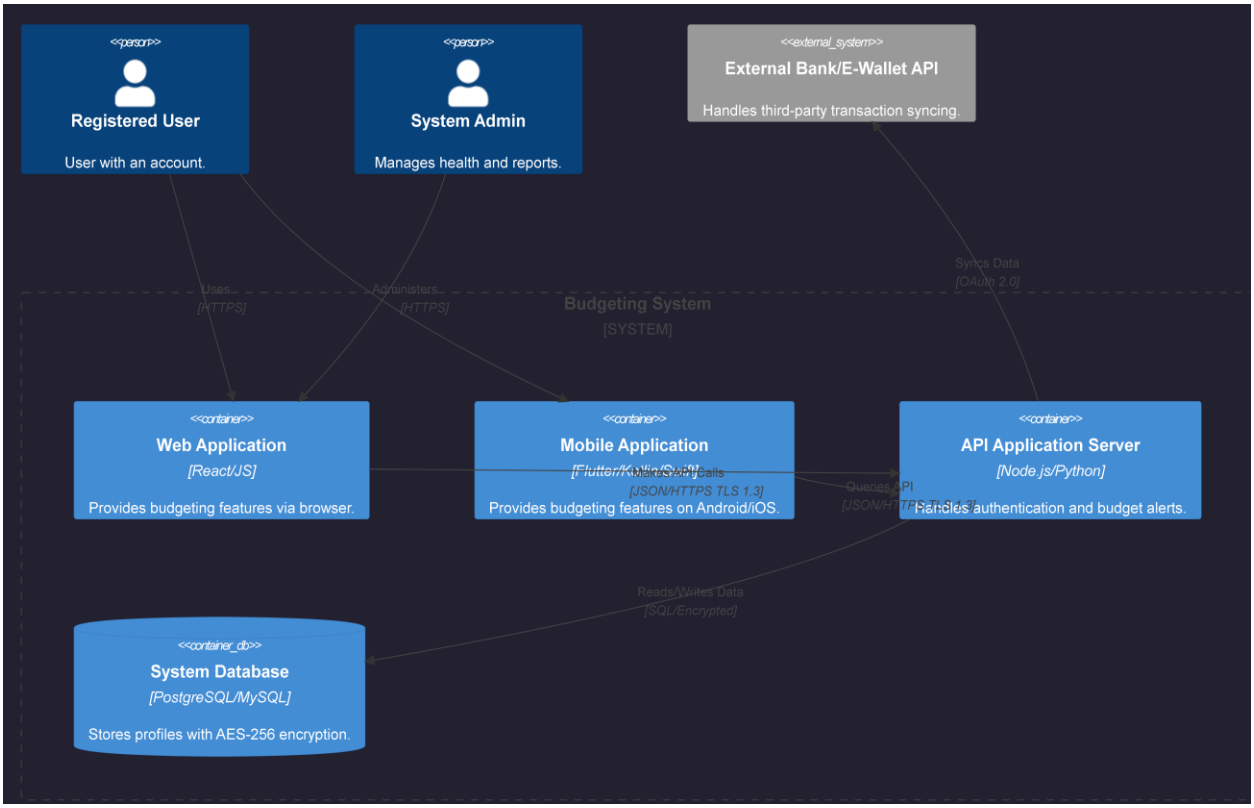
The most suitable software architecture is **3-Tier Layered Architecture** because

- **Scalability:** The SRS requires support for up to 10,000 simultaneous users. A 3-tier approach allows you to scale the server independently from the database.
- **Portability:** The system must support Android, iOS, and Web platforms. A central Application Tier (API) ensures all clients receive the same data and logic.
- **Maintainability:** The SRS specifically mentions the system should be **modular,** so updating one feature doesn't affect others.

System Components & Packages

Layer	Component/Package	Description
Presentation Tier	**Mobile/Web UI**	Handles user interaction, rendering dashboards, and charts.
Application Tier	**Auth & Profile Manager**	Manages User Sign-up, Login, and MFA/OAuth 2.0 security.
	Transaction Service	Processes income and expense entries and updates balances.
	Budget & Alert Engine	Monitors spending limits and triggers "Over-Limit" alerts.
	Analytics & Reporting	Generates the logic for pie charts and weekly summaries.
	External Sync Manager	Connects to Bank/E-Wallet APIs for auto-syncing.
Data Tier	**Relational DB**	Stores user profiles, hashed passwords, and transaction history.

CS251: Phase 1 – Digital Fin
Project: Budget wise
Software Requirements Specifications



II. Class Diagram(s)

1) Identify Classes per Component:

User Management

- User (Entity)
- Authentication manage (Entity)

Transaction Management

- Transaction (Entity)
- Income (Entity)
- Expense (Entity)
- Category (Entity)

Budget Management

- Budget (Entity)

Goal Management

- FinancialGoal (Entity)

Reporting

- Report (Entity)

Notification System

- Notification (Entity)

2) Divide into Packages (Subsystems)

UserManagement Package:
- User
- authentication manage

TransactionManagement Package:
- Transaction
- Income
- Expense
- Category

BudgetManagement Package:
- Budget

GoalManagement Package:
- FinancialGoal

Reporting Package:
- Report

Notification Package:
- Notification

3) Attributes & Operations

I. **User (Entity)**

Attributes:

- - userId: int
- - name: String
- - email: String
- - password: String
- - balance: double

Methods:

- + manageProfile(name: String, email: String): void

II. **Transaction (Entity)**

Attributes:

- - transactionId: int
- - userId: int
- - amount: double
- - date: Date
- - description: String
- - type: String

Methods:

- + addTransaction(): void
- + editTransaction(): void
- + validateAmount(amount: double): boolean

III. **Income (Entity)**

Attributes:

- - source: String

IV. **Expense (Entity)**

Attributes:

- - category: Category
- - paymentMethod: String
- - note: String

V. **Category (Entity)**

Attributes:

- - categoryId: int
- - userId: int (nullable)
- - name: String
- - type: String
- - isDefault: boolean

Methods:

- + createCategory(): void
- + deleteCategory(): void
- + seedDefaultCategories(): void

VI. **Budget (Entity)**

Attributes:

- - budgetId: int
- - amount: double
- - spentAmount: double
- - category: Category

Methods:

- + setBudgetPeriod(days: int): void
- + calculateRemaining(): double
- + checkLimit(): boolean

VII. FinancialGoal (Entity)

Attributes:

- - goalId: int
- - goalName: String
- - targetAmount: double
- - currentAmount: double
- - deadline: Date

Methods:

- + updateProgress(): void
- + calculatePercentage(): double
- + isAchieved(): boolean

VIII. Notification (Entity)

Attributes:

- - notificationId: int
- - message: String
- - isRead: boolean
- - messageTime: Date

Methods:

- + markAsRead(): void

IX. Report (Entity)

Attributes:

- - reportId: int
- - startDate: Date
- - endDate: Date

Methods:

- + generateReport(): void
- + generateChart(): void

X. **Authentication manage (Entity)**

Methods:

- + register(): user
- + login(): user

4) Core Models (Inheritance)

I. Transaction (Base Class)

- Attributes: userId: ObjectId, amount: Number, date: Date, description: String, type: String
- Methods: validateAmount(), addTransaction(), editTransaction()

II. Income (Inherits from Transaction)

- Attributes: source: String

III. Expense (Inherits from Transaction)

- Attributes: category: ObjectId, paymentMethod: Enum, note: String

5) Supporting Classes (Association)

I. Category

- Attributes: userId: ObjectId (Nullable), name: String, type: Enum, isDefault: Boolean
- Relation: One Category can be associated with many Expense objects.

II. TransactionService

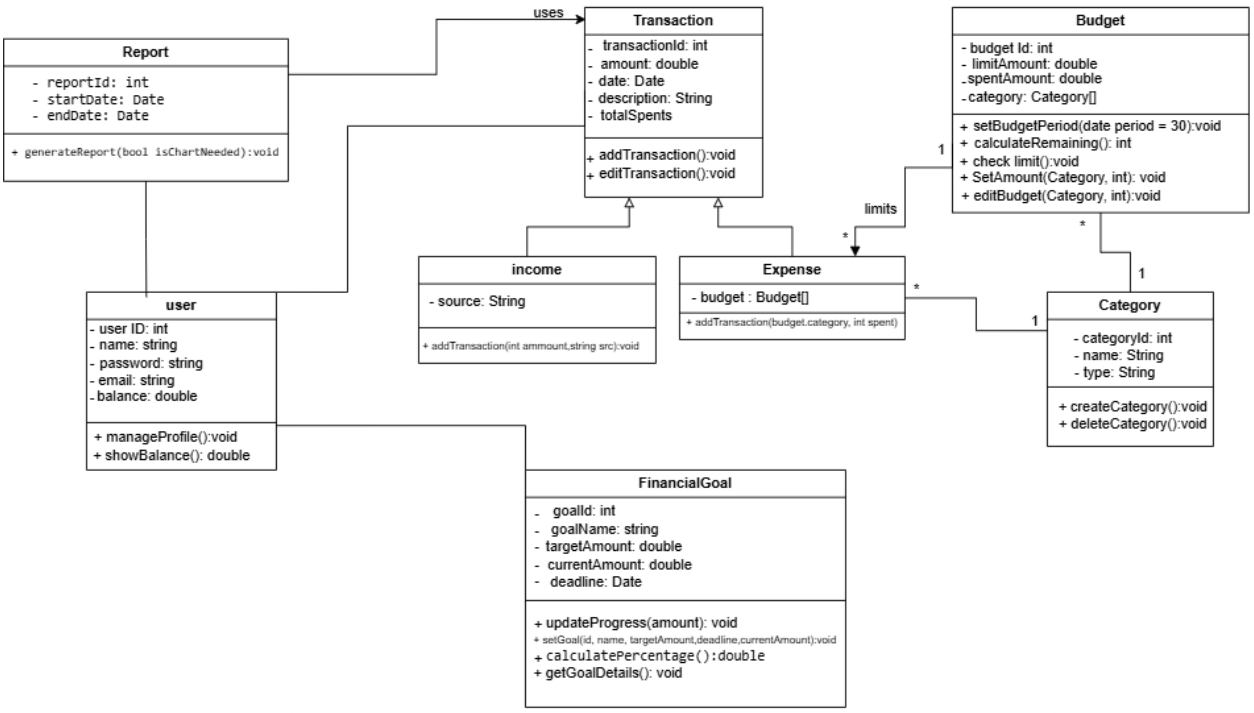
- Methods: addTransaction(), editTransaction(), deleteTransaction(), getAllTransactions(), calculateBalance()

III. TransactionController

- Methods: createTransaction(), getTransaction(), updateTransaction(), deleteTransaction(), getAllCategories()

CS251: Phase 1 – Digital Fin
Project: Budget wise
Software Requirements Specifications

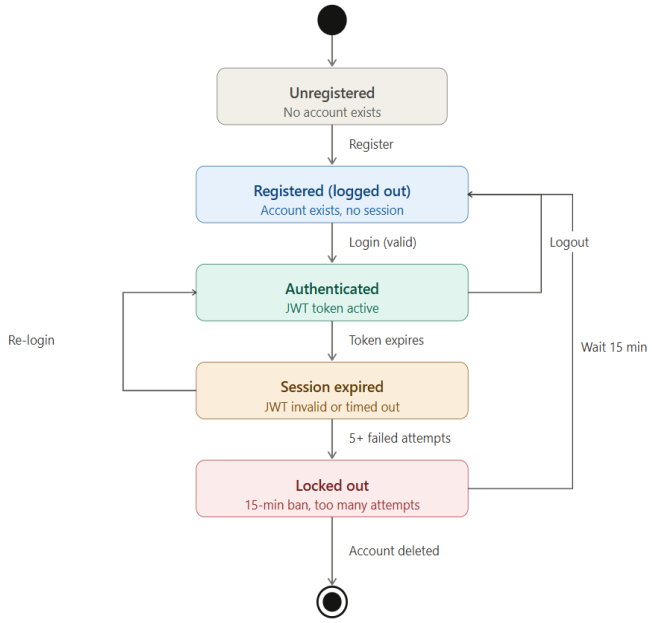
Class Diagram:



III. Class Descriptions

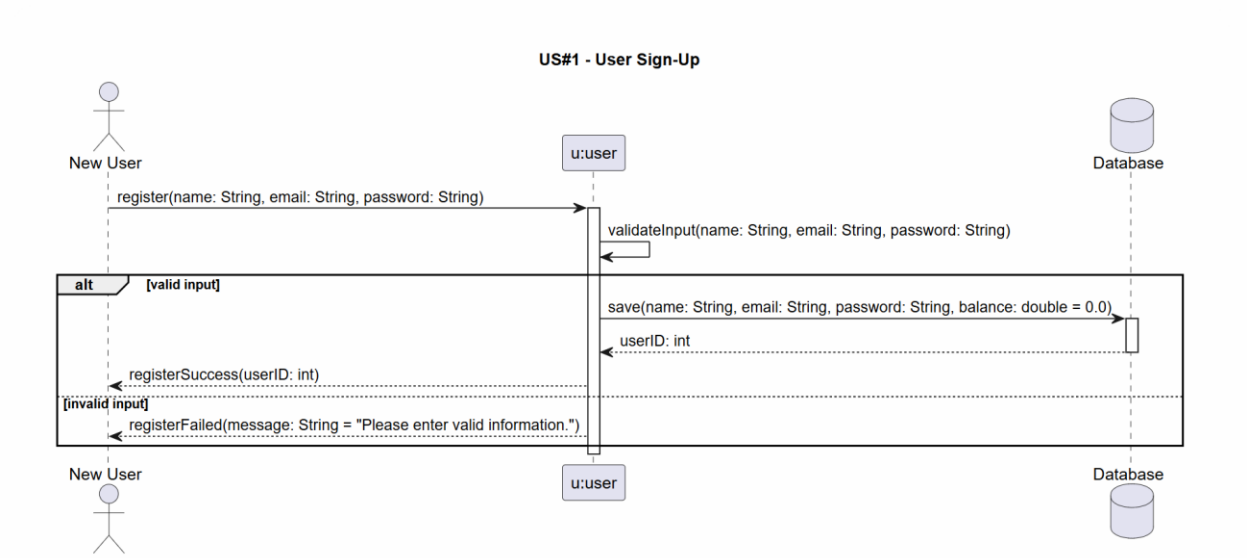
Class ID	Class Name	Description & Responsibility
1.	**User**	Users interact with the website to manage personal financial activities. The User is responsible for registering, logging in, managing their profile, creating and managing transactions, budgets, financial goals, reports, and reading the notifications.
2.	**Transaction**	Represents a financial operation performed by the user. It stores details such as amount, date, and description. The Transaction class is responsible for recording, updating, and maintaining financial data, and it serves as a base class for Income and Expense.
3.	**Income**	Represents money received by the user. It extends the Transaction class and adds information about the income source. It is responsible for increasing the user's balance and tracking incoming funds.
4.	**Expense**	Represents money spent by the user. It extends the Transaction class and is associated with a specific category. It is responsible for decreasing the user's balance and contributing to budget tracking and limit checking.
5.	**Category**	Represents a classification of expenses (and budgets) such as food or transport. It is responsible for organizing financial data, enabling categorization of expenses, and supporting budget allocation and tracking per category.
6.	**Budget**	Represents a spending limit defined by the user for a specific category over a certain period. It is responsible for tracking expenses against the defined limit, calculating the remaining amount, checking if the limit is exceeded, and triggering notifications when necessary.
7.	**Financial Goal**	Represents a savings goal set by the user with a target amount and deadline. It is responsible for tracking progress toward the goal, updating the current saved amount, and calculating the completion percentage.
8.	**Notification**	Represents a message or alert generated by the system, usually triggered by events such as exceeding a budget limit. It is responsible for informing the user about important updates and maintaining the read/unread status of notifications.
9.	**Report**	Represents a summary of the user's financial data over a selected period. It is responsible for generating reports and charts based on transaction data to provide insights into income, expenses, and financial behavior.
10.	**Authentication manage**	Handles user authentication processes such as registering new users and logging in existing ones. It ensures that user credentials are validated and provides access to the system after successful authentication.

IV.State Diagram For User Account:

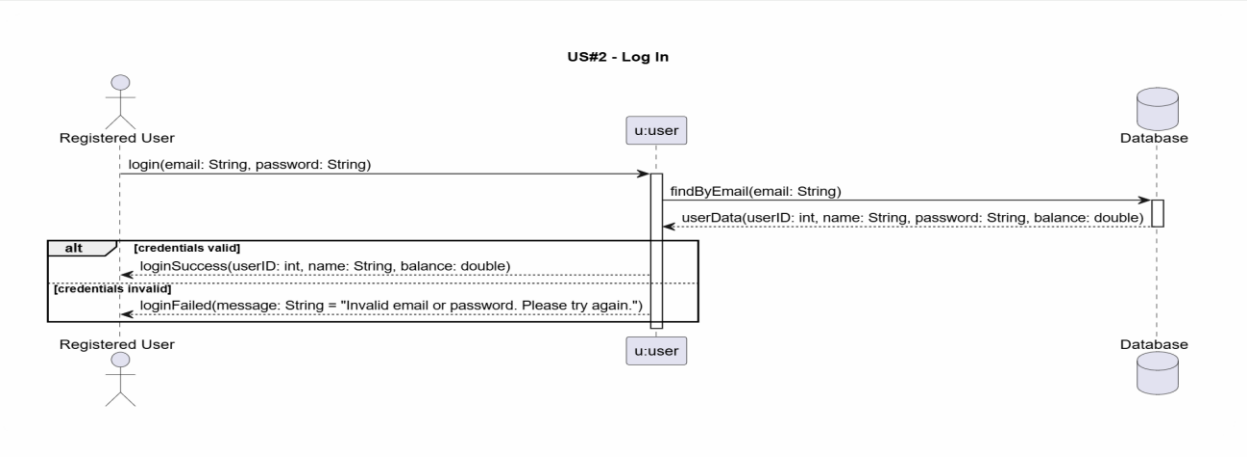


V. Sequence diagrams

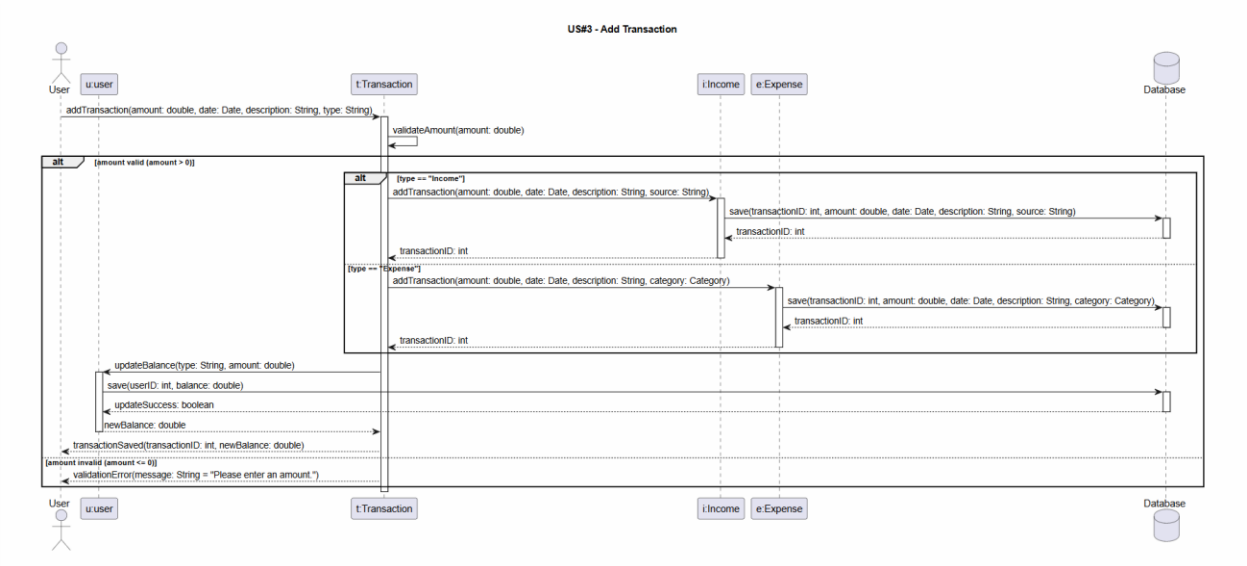
US#1- User Sign-Up



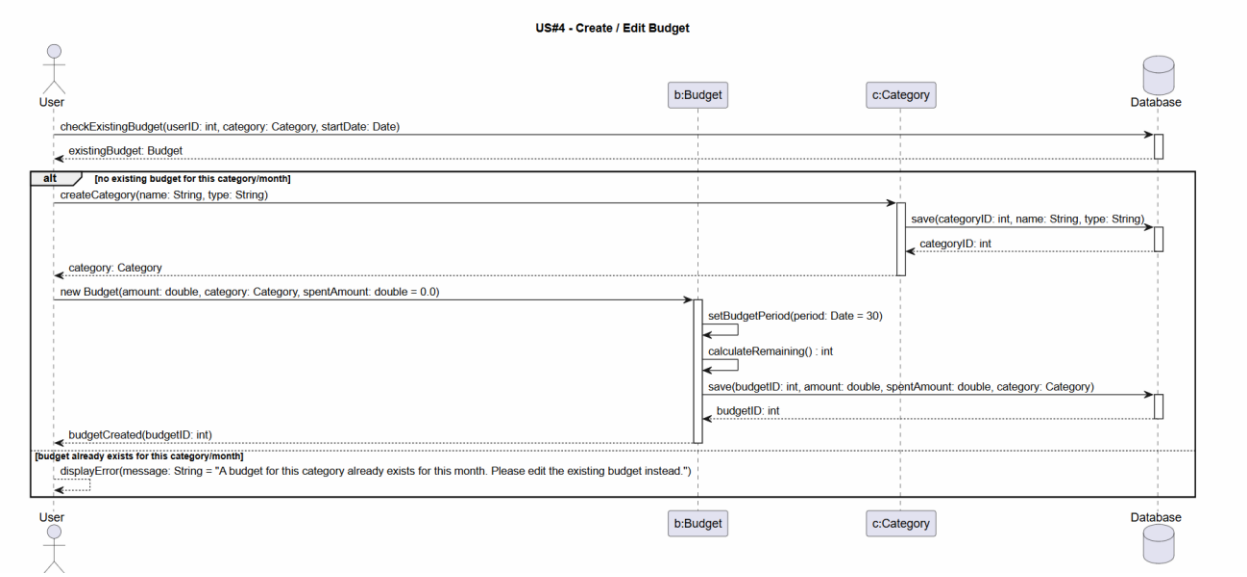
US#2-Log In



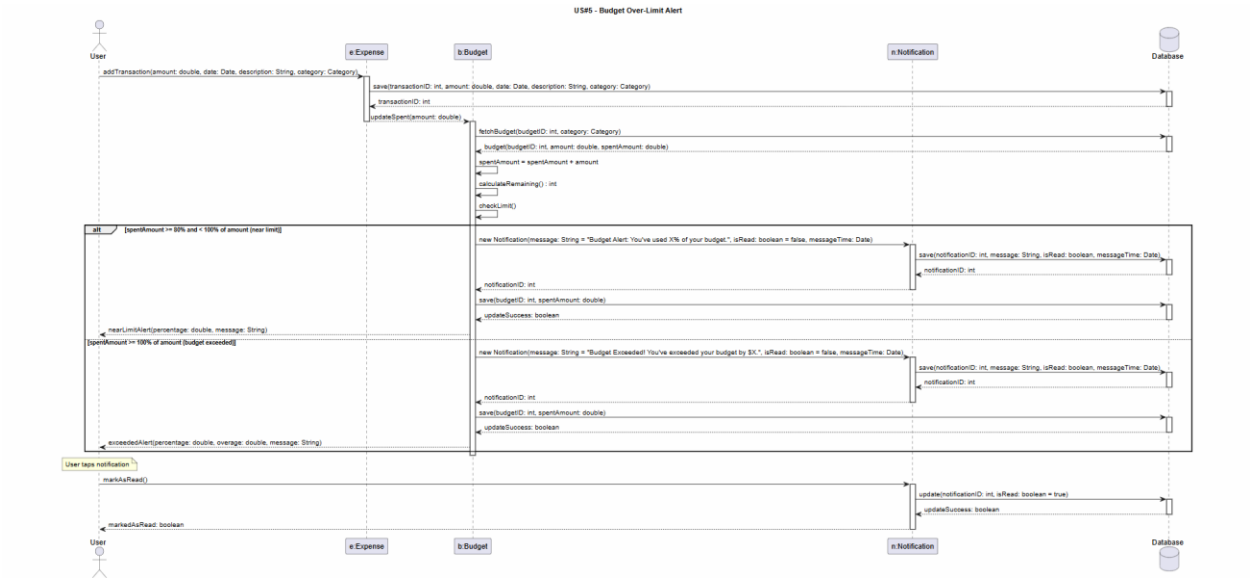
US#3-Add Transaction



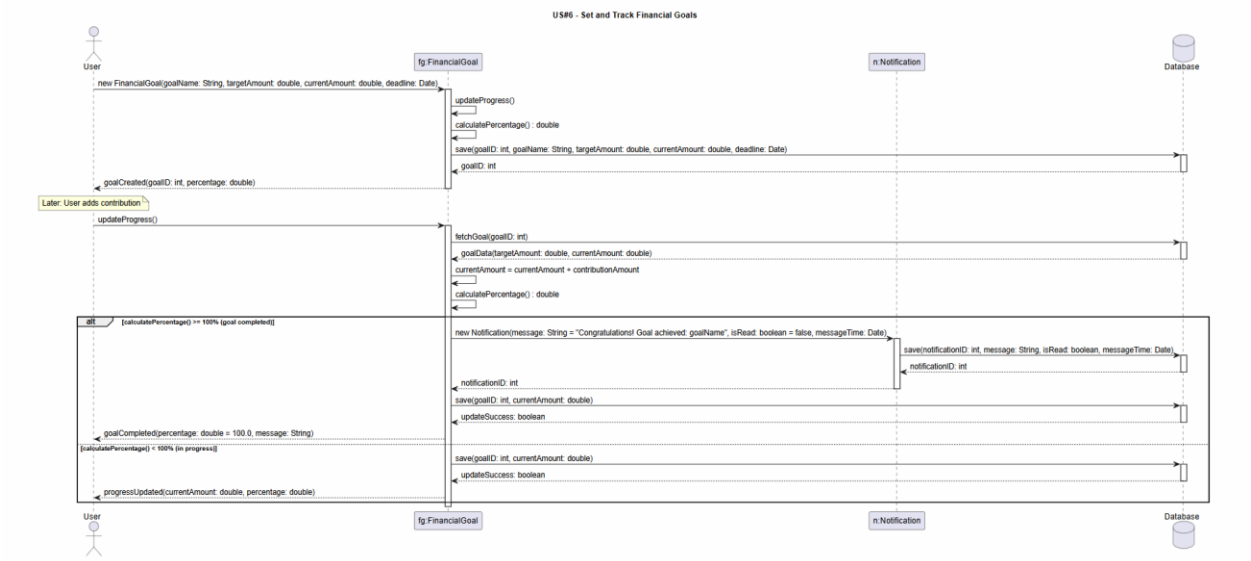
US#4-Create/Edit Budget



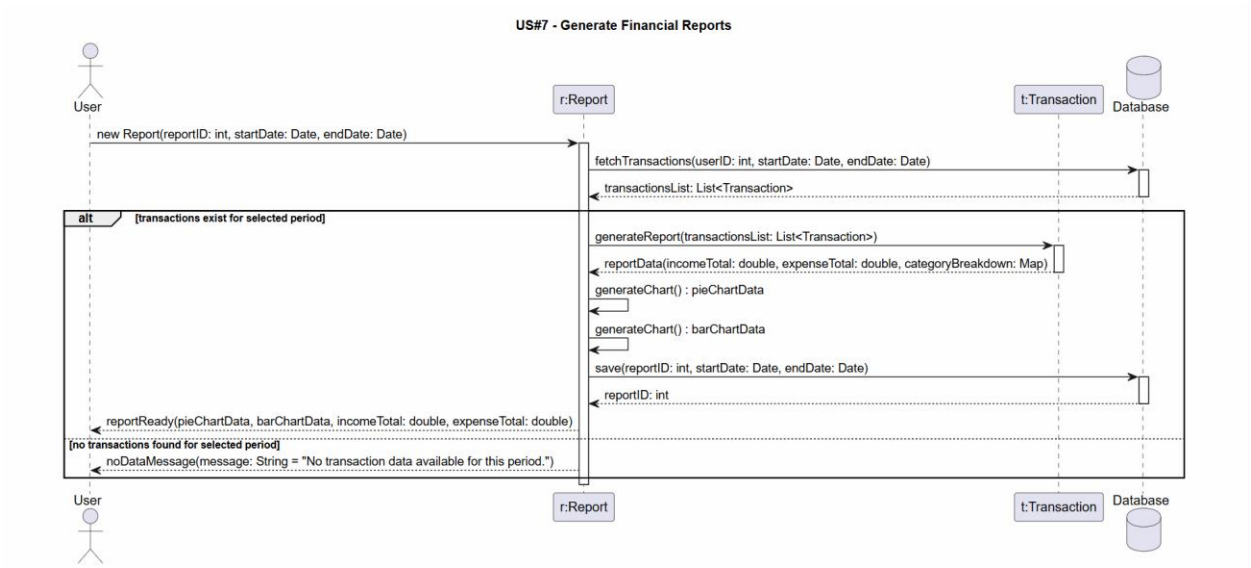
US#5-Budget Over Limit Alert



US#6- Set and Track Financial Goals



US#7-Generate Financial Reports


US#7 - Generate Financial Reports
User
r:Report
t:Transaction
Database
new Report(reportID: int, startDate: Date, endDate: Date)
fetchTransactions(userID: int, startDate: Date, endDate: Date)
transactionsList: List<Transaction>
alt
[transactions exist for selected period]
generateReport(transactionsList: List<Transaction>)
reportData(incomeTotal: double, expenseTotal: double, categoryBreakdown: Map)
generateChart() : pieChartData
generateChart() : barChartData
save(reportID: int, startDate: Date, endDate: Date)
reportID: int
reportReady(pieChartData, barChartData, incomeTotal: double, expenseTotal: double)
[no transactions found for selected period]
noDataMessage(message: String = "No transaction data available for this period.")
User
r:Report
t:Transaction
Database

Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
US#1 – User Sign-Up	User (actor)	register(name, email, password) validateInput(name, email, password) registerSuccess(userID) registerFailed(message)
	user	save(name, email, password, balance) userID : int
	Database	—
US#2 – Log In	Registered User (actor)	login(email, password) loginSuccess(userID, name, balance) loginFailed(message)
	user	findByEmail(email) userData(userID, name, password, balance)
	Database	—
US#3 – Add Transaction	User (actor)	addTransaction(amount, date, description, type) transactionSaved(transactionID, newBalance) validationError(message)
	user	updateBalance(type, amount) save(userID, balance) updateSuccess : boolean newBalance : double
	Transaction	validateAmount(amount) addTransaction(...) transactionID : int
	Income	addTransaction(amount, date, description, source) save(transactionID, amount, date, description, source) transactionID : int
	Expense	addTransaction(amount, date, description, category) save(transactionID, amount, date, description, category) transactionID : int
	Database	—
US#4 – Create / Edit Budget	User (actor)	checkExistingBudget(userID, category, startDate) new Budget(amount, category, spentAmount) budgetCreated(budgetID) displayError(message)
	Budget	setBudgetPeriod(period) calculateRemaining() : int save(budgetID, amount, spentAmount, category)

Sequence Diagram	Classes Used	All Methods Used
		budgetID : int
	Category	createCategory(name, type) save(categoryID, name, type) categoryID : int category : Category
	Database	—
US#5 – Budget Over-Limit Alert	User (actor)	addTransaction(amount, date, description, category) markAsRead() nearLimitAlert(percentage, message) exceededAlert(percentage, overage, message) markedAsRead : boolean
	Expense	save(transactionID, amount, date, description, category) updateSpent(amount) transactionID : int
	Budget	fetchBudget(budgetID, category) budget(budgetID, amount, spentAmount) spentAmount = spentAmount + amount calculateRemaining() : int checkLimit() save(budgetID, spentAmount) updateSuccess : boolean
	Notification	new Notification(message, isRead, messageTime) save(notificationID, message, isRead, messageTime) notificationID : int update(notificationID, isRead = true) updateSuccess : boolean
	Database	—
US#6 – Set and Track Financial Goals	User (actor)	new FinancialGoal(goalName, targetAmount, currentAmount, deadline) updateProgress() goalCreated(goalID, percentage) progressUpdated(currentAmount, percentage) goalCompleted(percentage, message)
	FinancialGoal	updateProgress() calculatePercentage() : double fetchGoal(goalID) goalData(targetAmount, currentAmount) currentAmount = currentAmount + contributionAmount save(goalID, goalName, targetAmount, currentAmount, deadline) save(goalID, currentAmount) goalID : int

Sequence Diagram	Classes Used	All Methods Used
		updateSuccess : boolean
	Notification	new Notification(message, isRead, messageTime) save(notificationID, message, isRead, messageTime) notificationID : int
	Database	—
US#7 – Generate Financial Reports	User (actor)	new Report(reportID, startDate, endDate) reportReady(pieChartData, barChartData, incomeTotal, expenseTotal) noDataMessage(message)
	Report	fetchTransactions(userID, startDate, endDate) generateReport(transactionsList) reportData(incomeTotal, expenseTotal, categoryBreakdown) generateChart() : pieChartData generateChart() : barChartData save(reportID, startDate, endDate) reportID : int
	Transaction	fetchTransactions(userID, startDate, endDate) transactionsList : List<Transaction>
	Database	—

SOLID PRINCIPLE:

S-Single Responsibilty Princible:

- Transaction (Model) → Only defines data structure. Changes only if the database schema changes.
- TransactionService → Only handles business logic. Changes only if business rules change.
- TransactionController → Only handles HTTP in/out. Changes only if API format changes.

O-Open/Closed Principle:

The Transaction class is open for extension but closed for modification.
Income and Expense extend Transaction using Mongoose discriminators without changing the base Transaction code.
If we need a new type (e.g., "Transfer"), we create a new discriminator file with the existing Transaction, Income, and Expense code that is never modified.

L-Liskov Substitution Principle:

Income and Expense objects can be used anywhere a Transaction is expected. Transaction.find() returns both Income and Expense objects. The getAllTransactions method works identically regardless of the sub-type. The Report feature can call the same query and process both types without knowing which is which.

Design Patterns:

1. MVC — Model View Controller

The MVC pattern separates the application into three main components, making the codebase easier to maintain, test, and scale. Each layer has a single responsibility.

Model: UserModel.ts — defines the User schema and interacts with MongoDB.

Controller: authController.ts — handles business logic: register, login, logout, and delete account.

Route (View): authRoutes.ts — maps HTTP requests to the correct controller function.

2. Middleware Pattern

Middleware functions intercept HTTP requests before they reach the controller. In Budget-Wise, the verifyToken middleware protects sensitive routes by validating the JWT token on every request. If the token is missing or invalid, the middleware rejects the request with a 401 Unauthorized response — the controller is never reached.

Request → verifyToken (middleware) → Controller → Response

Applied to protected routes: `router.post("/logout", verifyToken, logout) | router.get("/profile", verifyToken, profileHandler) | router.delete("/delete-account", verifyToken, deleteAccount)`

3. Repository Pattern

The Repository pattern abstracts all database operations behind the Model layer. Controllers never write raw database queries — they call Model methods instead. This makes the code easier to test and allows the database to be swapped without changing business logic.

Database calls used in authController.ts: `User.findOne({ email }) | User.create({ name, email, password }) | User.findByIdAndDelete(userId)`

Tools

https://app.diagrams.net/ : for Class diagram

https://mermaid.live/ : for the Architecture diagram

PlantUML : for Secuence Diagram

Ownership Report

Item	Owners
Yasmin Mohamed Nabil	**Part of class diagram**
Lojain Mohamed Abu Elhassan	**Part of Sequence diagram**
Mario Nasser Fawzy	**Part of Architecture diagram**